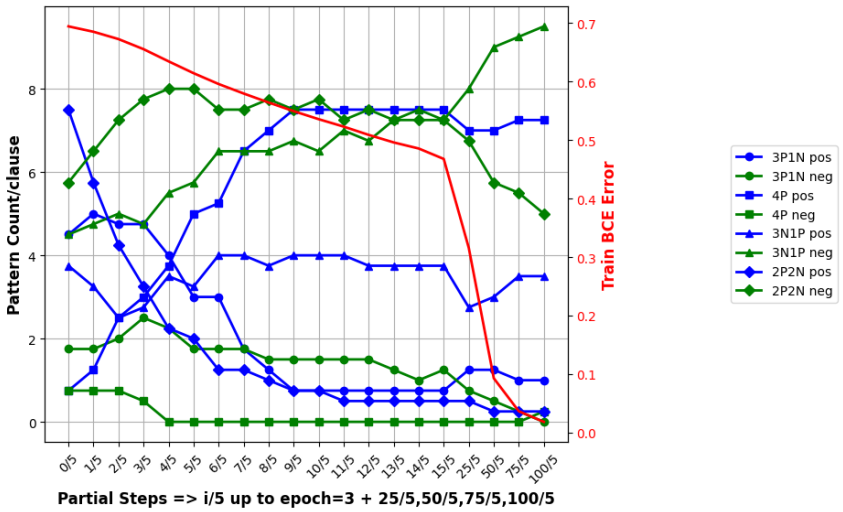

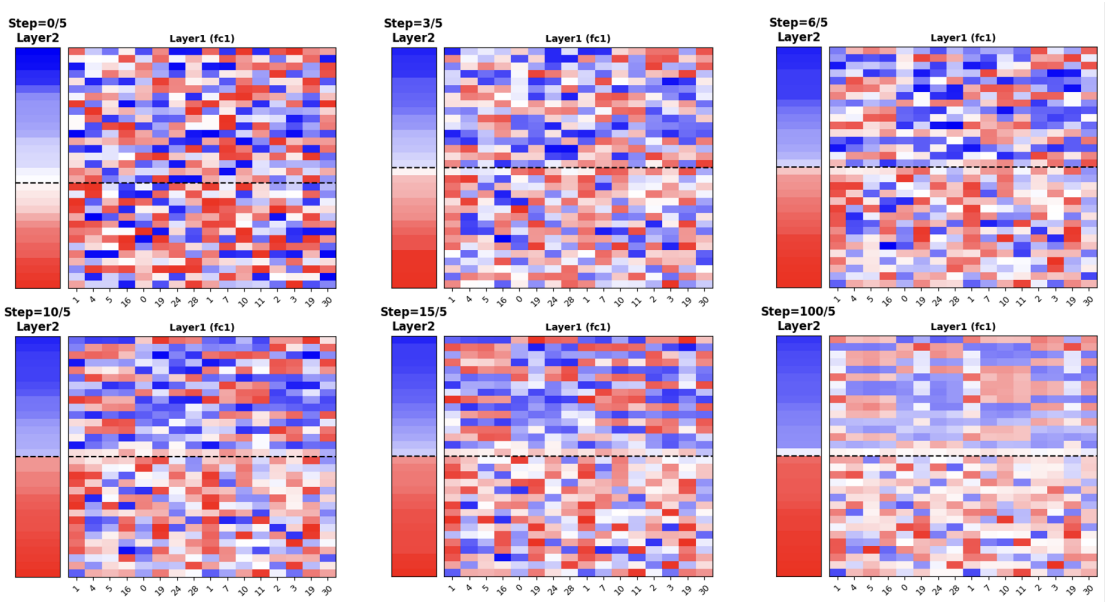

Figure 12: How codes emerge during training. For clarity, the 20 epochs are not uniformly distributed. Notice in the heat maps of the Layer 1 weights how error decreases as 4P prevelance increases. One can see the emergence of the 4P patterns in the positive witness neurons, and 3N1P in the negative witness neurons. The values in the graph are an average over 10 runs, but the heat maps are for a single run.

we did the same analysis of patterns types for clauses as we introduced above, but we are now

measuring the frequency of the pattern types during the training process. The x-axis shows the number of training epochs, where we split the first 3 epochs into 5 sub-epochs each to allow finer tracking at the start of the training. Thus, the x-axis is not linear. The graph also shows, in red, the training error as the training process proceeds. The patterns that emerge are intriguing. Note that the starting points of the curves are all about where we would expect through randomly setting each matrix entry to either positive or negative with equal probability (which is consistent with our initialization of the training process). As a result, the training starts with the 2P2N patterns as the most prevalent in both positive and negative rows. We then see that the network clearly decreases all the positive patterns apart from positive 4P which rapidly increases. Similarly, all the negative patterns apart from 3P1N and 2P2N decrease. This continues until epoch 2, from which point there seems to be little adjustment to the main 4P positive pattern but 2P2N negative starts to transition to 3N1P negative to deliver the final accuracy.

The final distribution of patterns is similar to what we have seen before, such as in Figure 7. In Figure 12, we also depict the heat maps that emerge at the various stages of this training (the graph is an average over 10 runs, but the heat maps are for a single run). The pattern distributions leave us with a clear understanding of what to expect in the heat maps: the final result is many 4P patterns as positive witnesses of the clauses and 3P1N patterns as negative witnesses (both appearing more than random), and also many 2P2N negative witnesses, though like all other patterns, they are less than or equal to the number that we would expect to appear at random. One can see that the 4P patterns form clear feature channel codes from step 6.

One of the most interesting aspects of the graph is the 4P positive curve, which has increased significantly before even the first epoch has completed, and in fact does not increase after the second epoch. This makes it clear that the 4P codes are an important part of how the model solves the problem and that gradient descent has a relatively easy time of finding them. More generally, the positive coding patterns converge to their desired result relatively quickly. However, at the point where those positive rows are stable (around the point labeled 10/5), the error is still around 55%. This was surprising to us, since we will see below that in some cases it is possible to get a very good approximation to the learning of the Boolean formula using only the positive rows of the weight matrix. But in the specific case studied here, the negative rows seem to be crucial. In those rows, 2P2N gets off to a fast start, but the system eventually realizes that 3N1P is a better pattern to use, and it isn't until 3N1P replaces much of the 2P2N that the training error actually disappears.

4.7 Clauses With a Negated Variable

So far we have shown how feature coding appears in DNFs that have all positive variables. In this section we show how features with negative variables are coded, by studying DNF clauses with 3 non-negated variables and 1 negated variable. We will see that with this slightly more complex formula, the overall coding behavior is very similar to the fully positive case. We also use this example to show how computation with feature channel codes proceeds.

We trained the same simple network with a single hidden layer as in our prior experiments, here with 32 input Boolean variables and 16 hidden neurons. In the top part of Figure 13, we show the result of training the model with the formula $(x_8 \wedge x_{17} \wedge x_{20} \wedge \neg x_{28}) \vee (\neg x_{10} \wedge x_{19} \wedge x_{22} \wedge x_{23})$. For legibility we are only showing the columns corresponding to the 8 variables in the 2 clauses, not all 32 of them. The input features are sorted in the order of the clauses. A quick glace at the matrix shows us the feature channel coding patterns. As before, the network has converged to have a negative bias in Layer 1 neurons corresponding to positive witnesses (positive Layer 2

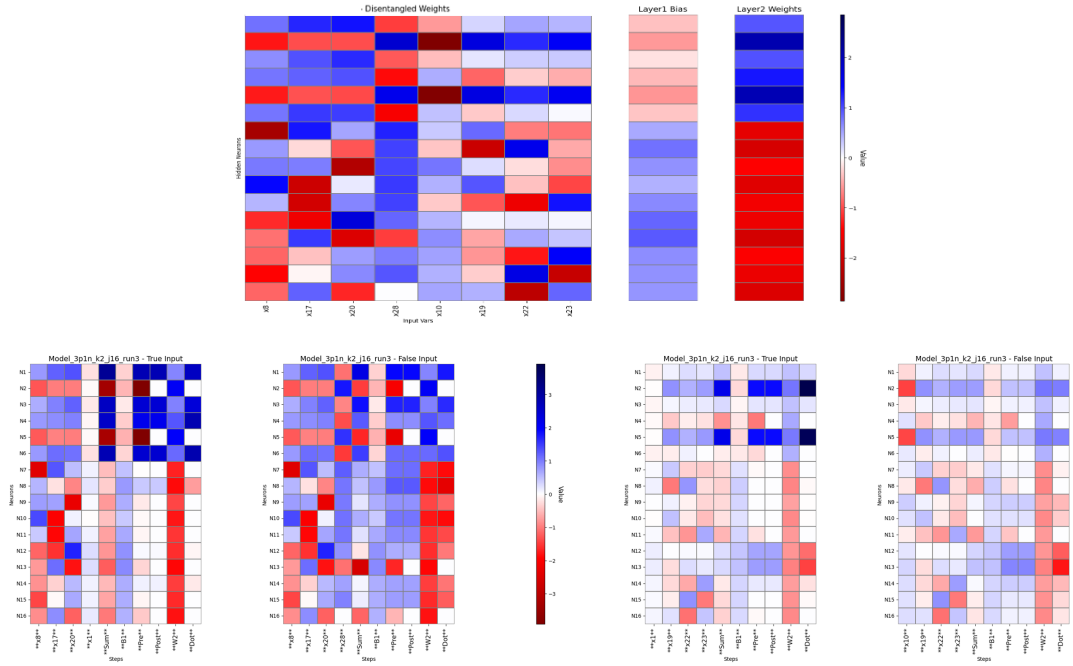

Figure 13: Execution traces show how channel coding in a model with clauses that have a negated variable each deliver the appropriate positive or negative response.

weights), and positive Layer 1 bias in neurons corresponding to negative witnesses (negative Layer 2 weights). Looking at the Layer 1 matrix entries for the clause $(x_8 \wedge x_{17} \wedge x_{20} \wedge \neg x_{28})$, we see clear positive coding rows emerge in neurons 1, 3, 4 and 6. For the clause $(\neg x_{10} \wedge x_{19} \wedge x_{22} \wedge x_{23})$, the positive coding rows are 1, 2, 3, and 5. Thus, the positive codes overlap in neurons 1 and 3, but the coding for the second clause has smaller magnitude weights in those overlapping rows (which is not a coincidence - see below.) The Boolean computation performed using these feature channels is as expected: the negated variables have corresponding columns with negative weights while the columns corresponding to positive variables have positive weights. As before we call this pattern 3P1N.

The bottom part of Figure 13 presents four execution traces for the model evaluating two clauses: $(x_8 \wedge x_{17} \wedge x_{20} \wedge \neg x_{28})$ (left two traces) and $(\neg x_{10} \wedge x_{19} \wedge x_{22} \wedge x_{23})$ (right two traces). Each trace details network activation over time (left-to-right), showing initial True variable inputs (first 4 columns), intermediate Layer 1 computations (dot product, bias, pre/post-ReLU values), Layer 2 weights, and the final Layer 1 neuron contributions to the Layer 2 output.

For the first clause, Trace 1 shows a satisfied case (x_8, x_{17}, x_{20}, x_1 True and x_{28} (not shown) False). Positive neurons (1, 3, 4, 6) activate according to x_8, x_{17}, x_{20}, while negative neurons (7-17) remain mostly inactive, resulting in a correct sigmoid output of 0.9999 (True). Trace 2 shows an unsatisfied case ($x_8, x_{17}, x_{20}, x_{28}$ True). Here, x_{28} inhibits positive neurons (1, 3, 4, 6) via negative weights and activates negative neurons (7-17) via positive weights, resulting in a correct sigmoid output of 0.0015 (False).